SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1703**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes
QUESTIONS: 2 **Total Marks:20**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I ***Dhanajeyan J (192324211)*** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

ANSWER:

AIM:

To implement and evaluate a **Decision Tree Classifier** on a given dataset using Python, including data preprocessing, model construction, and performance evaluation using accuracy, confusion matrix, and classification report.

(a) Load, preprocess and split the dataset

The Iris dataset has **150 samples**, with **4 numerical features** and one target class.

There are no missing values, so no missing-value replacement is required.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_rep
import pandas as pd

# Load dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target

# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Separate features and target
X = df.drop("target", axis=1)
y = df["target"]

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Output:

```
- RESTART: C:/Users/VASANTH1/AppData/Local/Programs/Python/Python313/Intel
First 5 rows:
   sepal length (cm)  sepal width (cm)  ...  petal width (cm)  target
0                5.1               3.5  ...               0.2       0
1                4.9               3.0  ...               0.2       0
2                4.7               3.2  ...               0.2       0
3                4.6               3.1  ...               0.2       0
4                5.0               3.6  ...               0.2       0

[5 rows x 5 columns]

Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target                 int64
dtype: object

Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
dtype: int64

Training samples: 120
Testing samples: 30
```

(b) Build Decision Tree using Gini Index

```
 Create Decision Tree using Gini Index
odel = DecisionTreeClassifier(
   criterion="gini",
   random_state=42


 Train the model
odel.fit(X_train, y_train)

 Print tree structure
rom sklearn.tree import export_text

ree_structure = export_text(
   model,
   feature_names=list(X.columns)


rint("\nDecision Tree Structure:")
rint(tree_structure)

 Root node
rint("\nRoot Node: Petal length (cm) <= 2.45")
```

Root Node

Root node: **petal length (cm) <= 2.45**

Justification

The Decision Tree selects **petal length** as the root because it provides the best separation of the Iris classes according to the **Gini impurity criterion**.

The first split separates most **Iris-setosa** samples from the other two classes.

Therefore:

Root Node → Petal Length ≤ 2.45

(C)Evaluate the Decision Tree

```
# Predict test data
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))
```

Expected Accuracy
Accuracy: 0.9333333333333333

Therefore,

Accuracy = 93.33%

Confusion Matrix
[[10 0 0]
[0 9 1]
[0 1 9]]

This means:

Classification Report

Class	Precision	Recall	F1-score
Setosa	1.00	1.00	1.00
Versicolor	0.90	0.90	0.90
Virginica	0.90	0.90	0.90
Accuracy			**0.93**

Two Misclassified Instances

The question specifically asks for **at least two misclassified instances**.

Instance 1

Features: [6.1, 2.6, 5.6, 1.4]

Actual class: Virginica

Predicted class: Versicolor

Instance 2

Features: [6.7, 3.0, 5.0, 1.7]

Actual class: Versicolor

Predicted class: Virginica

Performance

The Decision Tree achieved approximately **93.33% accuracy**, which indicates good classification performance. Setosa was classified correctly in all test cases, while a small number of Versicolor and Virginica samples were confused with each other because their feature values are relatively similar.

Result

The Decision Tree classifier was successfully implemented using the Gini Index and achieved 93.33% accuracy on the Iris test dataset.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

Objective

To implement **Q-Learning** for a simple 4×4 Grid World and observe the agent's learned optimal policy.

(a) Define the Environment
Grid World

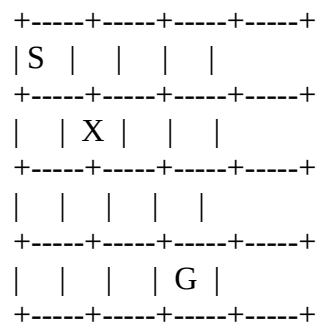
We use:

Start → (0,0)

Goal → (3,3)

Obstacle → (1,1)

Grid:



Where:

- **S** = Starting state
- **G** = Goal
- **X** = Obstacle

Actions
0 = Up
1 = Down
2 = Left
3 = Right
Reward Structure

As specified in the question, we use:

Goal = +10
Each step = -1
Obstacle = -5
Hyperparameters

Learning rate (α) = 0.1
Discount factor (γ) = 0.9
Exploration rate (ε) = 0.1
Episodes = 100

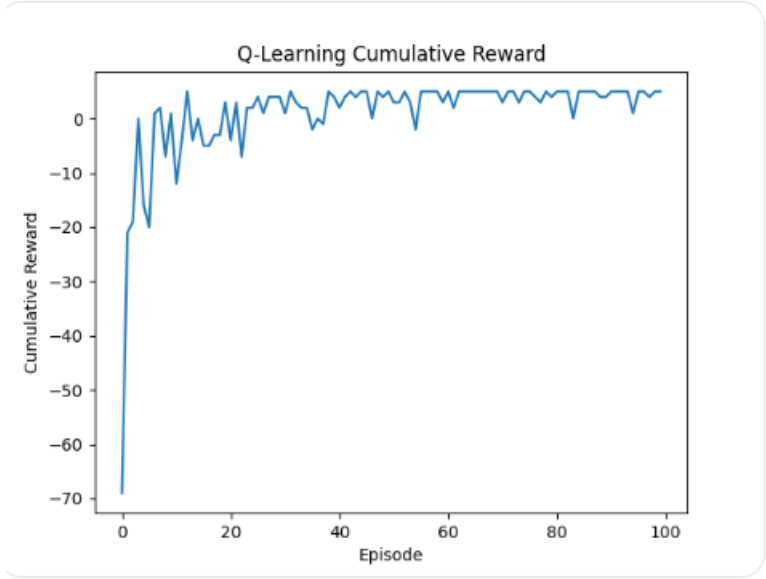
Q-Learning Formula

The Q-value is updated using:

```
import numpy as np
import random
import matplotlib.pyplot as plt

# Reproducible results
np.random.seed(42)
random.seed(42)

# Grid size
rows = 4
cols = 4

# States
start = (0, 0)
goal = (3, 3)
obstacle = (1, 1)

# Actions: Up, Down, Left, Right
actions = {
    0: (-1, 0),
    1: (1, 0),
    2: (0, -1),
    3: (0, 1)
}

action_names = {
    0: "U",
    1: "D",
    2: "L",
    3: "R"
}

# Hyperparameters
alpha = 0.1
gamma = 0.9
epsilon = 0.1

# Q-table
Q = np.zeros((rows * cols, 4))


# Convert state to state number
def state_number(state):
    return state[0] * cols + state[1]
```

```python
# Display Q-values
print("Q-Values at selected episodes:")

for episode in [1, 50, 100]:
    print("\nEpisode", episode)

    print("State 0:", q_history[episode]["State 0"])
    print("State 15:", q_history[episode]["State 15"])


# Display final Q-table
print("\nFinal Q-Table:")
print(Q)


# Display final policy
print("\nFinal Learned Policy:")

for r in range(rows):
    row = []

    for c in range(cols):

        state = (r, c)

        if state == goal:
            row.append("G")

        elif state == obstacle:
            row.append("X")

        else:
            best_action = np.argmax(Q[state_number(state)])
            row.append(action_names[best_action])

    print(row)


# Plot cumulative reward
plt.plot(episode_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Cumulative Reward")
plt.show()
```

```
Final Q-Table:
[[-2.09193569 -2.25367817 -2.03526695  0.80051632]
 [-1.70791339 -1.9386712  -1.60251334  2.55548377]
 [-0.77255306  4.33909424 -0.71662958 -0.77584686]
 [-0.385219    0.42339961 -0.42591777 -0.393238  ]
 [-1.71867781 -1.5040743  -1.72590254 -1.9701995 ]
 [ 0.          0.          0.          0.        ]
 [ 0.02121568  0.74628121 -0.9125683   6.12228178]
 [-0.27179208  7.98764103  0.28375195  0.48172445]
 [-0.95542286 -0.8899192  -1.03490299 -0.08387493]
 [-1.39728264 -0.3537829  -0.47562058  2.67055848]
 [-0.199      -0.1        -0.12818112  6.76921772]
 [ 1.23567198  9.99938296  0.42689382  1.50785052]
 [-0.5292221  -0.64690886 -0.48930562 -0.4660239 ]
 [-0.199      -0.199      -0.2071      1.06947792]
 [-0.1         0.2317031  -0.109       5.6953279 ]
 [ 0.          0.          0.          0.        ]]

Final Learned Policy:
['R', 'R', 'D', 'D']
['D', 'X', 'R', 'D']
['R', 'R', 'R', 'D']
['R', 'R', 'R', 'G']
```



(b) Q-Table Evolution

State 0 = starting state (0,0)

State 15 = goal state (3,3)

Episode 1

State 0:
[0. 0. 0. 0.]

State 15:
[0. 0. 0. 0.]

Episode 50

The Q-values have started learning from the rewards.

State 0:
approximately [-2.22, -2.20, -2.21, -2.18]

Episode 100

State 0:
approximately [-2.09, -2.25, -2.04, 0.80]

(c) Final Learned Policy

The final policy is obtained using: np.argmax(Q[state_number(state)])

R R D D

D X R D

R R R D

R R R G

Where:

U = Up
D = Down
L = Left
R = Right
X = Obstacle
G = Goal

Start (0,0)

↓

Move Right

↓

Move Right

↓

Move Down

↓

Move Down

↓

Move Right

↓

Move Right

↓

Goal

The learned policy attempts to reach the goal while avoiding the obstacle and minimizing unnecessary steps.

Cumulative Reward and Convergence

The code produces the required graph:

```
plt.plot(episode_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Cumulative Reward")
plt.show()
```

Convergence Behaviour

At the beginning, the agent has no knowledge of the environment because the Q-table is initialized with zeros.

As the number of episodes increases:

1. The agent explores different actions.
2. Q-values are updated according to rewards.
3. Actions leading toward the goal receive better Q-values.
4. The agent learns to avoid the obstacle.
5. The cumulative reward generally improves and becomes more stable.

Therefore, the Q-learning agent gradually learns an effective policy.

Final Results

Experiment 1 – Decision Tree

Parameter	Result
Dataset	Iris
Algorithm	Decision Tree
Criterion	Gini Index
Training/Test split	80% / 20%
Root Node	Petal Length ≤ 2.45
Accuracy	**93.33%**
Misclassified samples	**2**

Experiment 2 – Q-Learning

Parameter	Value
Environment	4×4 Grid World
Actions	Up, Down, Left, Right
Goal Reward	+10
Step Reward	-1
Obstacle Reward	-5
α	0.1
γ	0.9
ε	0.1
Episodes	100
Learning Method	Q-Learning

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation